Foundations of Computing Systems Laboratory
Laboratory Test 2

Date: 15 October 2026 **Time:** 2:00 p.m.–5:00 p.m.
Duration: 3 hours **Maximum marks:** 50

Name: _____ **Roll number:** _____

General instructions

- All five questions are compulsory. This is an individual examination.
- This test concerns only the translation of VM programs into Hack assembly. You are *not* required to translate Hack assembly into machine code.
- Use the Hack assembly-language reference at the end of this paper.
- Unless a question states otherwise, every stand-alone generated program must begin by setting the stack pointer to 256.
- For question i, you need to create a file `VMTranslatori.py` that translates the supplied `.vm` file into a Hack assembly-language file. For example, for question 1, you will create `VMTranslator1.py` that translates file `PushConstants.vm` into `PushConstants.asm`.

Question 1: Pushing Constants onto the Stack [10 marks]

The stack pointer, `SP`, is stored in `RAM[0]`. Initially set `RAM[0]` to 256. The value of `SP` is the address of the next free stack location. Therefore, pushing a value requires storing it in `RAM[SP]` and then incrementing `SP`.

Use the supplied file:

```
// PushConstants.vm
push constant 17
push constant 4
push constant 29
push constant 0
```

After execution, the supplied test checks

$$\texttt{RAM[256]} = 17, \quad \texttt{RAM[257]} = 4,$$
$$\texttt{RAM[258]} = 29, \quad \texttt{RAM[259]} = 0, \quad \texttt{RAM[0]} = 260.$$

The file above is only one example. Your translator will also be tested with `.vm` files containing different numbers of `push constant i` commands and different constant values. It must place every value in successive stack locations and leave `SP` pointing to the next free location. Do not hard-code the commands or expected values shown above.

Submit: `PushConstants.asm` and the translator `VMTranslator1.py`.

Question 2: Addition on the Stack [10 marks]

Extend the translator to implement `add`. The command pops the top value y, pops the next value x, and pushes $x + y$. The final value of `SP` must reflect that two operands have been replaced by one result.

```
// Add.vm
push constant 17
push constant 25
add
```

After execution, the supplied test checks that `RAM[256]=42` and `RAM[0]=257`.

The constants 17 and 25 are only example inputs. Your translator will also be tested with other pairs of constants. Do not hard-code these values or their sum; the generated assembly must add whichever two values are pushed by the VM program.

Submit: `Add.asm` and the translator `VMTranslator2.py`.

Question 3: Accessing the Temp Segment [10 marks]

Extend the translator to implement `push temp i` and `pop temp i`. The VM temp segment is mapped as follows:

$$\texttt{temp 0}, \ldots, \texttt{temp 7} \quad \longleftrightarrow \quad \texttt{RAM[5]}, \ldots, \texttt{RAM[12]}.$$

The supplied test places two input values in `R11` and `R12`. Translate the following program, which must add them and store the result in `R10`:

```
// TempAdd.vm
push temp 6          // push RAM[11]
push temp 7          // push RAM[12]
add
pop temp 5           // pop the result into RAM[10]
```

The test runs the program with several pairs of values in `RAM[11]` and `RAM[12]`. For each pair it checks that `RAM[10]=RAM[11]+RAM[12]` and that `RAM[0]=256` after the result has been popped.

Submit: `TempAdd.asm` and the translator `VMTranslator3.py`.

Question 4: Program Flow—Sum of the First n Numbers [15 marks]

The following program reads n from `RAM[10]` (`temp 5`), computes $1+2+\cdots+n$, and writes the result to `RAM[11]` (`temp 6`).

```
// SumN.vm
push temp 5
pop temp 0           // counter = n
push constant 0
pop temp 1           // sum = 0

label LOOP
push temp 0
push constant 0
eq                   // pop y and x; push -1 if x=y, otherwise push 0
if-goto END          // pop the result of eq; jump if it is nonzero
push temp 1
push temp 0
add
pop temp 1           // sum = sum + counter
push temp 0
push constant 1
sub
pop temp 0           // counter = counter - 1
goto LOOP
```

```
label END
push temp 1
pop temp 6          // RAM[11] = sum
```

Extend the translator to implement `sub`, `eq`, `label`, `goto`, and `if-goto`. The Hack VM Boolean convention is: **true is** -1 (all 16 bits are 1), and **false is 0**. Therefore, `eq` must push -1 when its operands are equal and 0 otherwise. Use unique internal assembly labels when translating comparisons.

Translate and test the program for $n = 0, 1, 5, 10$. The expected values in `RAM[11]` are $0, 1, 15, 55$.

Submit: `SumN.asm` and the translator `VMTranslator4.py`.

Question 5: Recursion and Return [15 marks]

The single supplied file `RecursiveSum.vm` contains two functions. The main function reads n from `RAM[10]`, copies it to `temp 0`, calls a recursive function, and stores the returned value in `RAM[11]`. No arguments are passed to either function.

```
function Main.main 0
push temp 5
pop temp 0          // temp 0 holds the current value of n
call Main.sum 0
pop temp 6
label END
goto END

function Main.sum 0
push temp 0
push constant 2
lt
if-goto BASE
push temp 0         // save the current n on the stack
push temp 0
push constant 1
sub
pop temp 0          // temp 0 = n-1 for the recursive call
call Main.sum 0
add                 // saved n + returned sum
return

label BASE
push temp 0         // return 0 or 1
return
```

The function uses `temp 0` for its current input. It saves n on the stack before replacing `temp 0` by $n - 1$; after the recursive return, `add` combines the saved n and returned sum.

For this question, use the following **simplified calling convention**. There are no arguments, local variables, or saved memory segments. A call reserves one stack location for the return value, pushes a unique return address, and jumps to the function. Thus, the relevant stack states are:

$$\begin{aligned} \text{before call} &: \cdots \mid \leftarrow \texttt{SP} \\ \text{inside function} &: \cdots \mid \text{return slot} \mid \text{return address} \mid \leftarrow \texttt{SP} \\ \text{after return} &: \cdots \mid \text{return value} \mid \leftarrow \texttt{SP} \end{aligned}$$

Complete the translation of:

```
function functionName 0
call functionName 0
return
```

Requirements:

1. Emit bootstrap code that sets `SP=256` and calls `Main.main 0`.
2. Translate `function f 0` by declaring the assembly label for `f`.
3. Scope labels to their functions. For example, translate `BASE` inside `Main.sum` to an assembly label such as `Main.sum$BASE`.
4. Every `call` must reserve one stack slot for the return value, push a unique return-address label, and jump to the function. Do not push `LCL`, `ARG`, `THIS`, or `THAT`.
5. Immediately before `return`, the function's result is at the top of the stack. The return code must preserve this result, recover the return address, place the result in the reserved return-value slot, leave `SP` pointing just after that value, and jump to the recovered address. Scratch registers such as `R13` and `R14` may be used.

Produce `RecursiveSum.asm`. The supplied test uses $n = 0, 1, 5, 10$ and expects `RAM[11]` to contain $0, 1, 15, 55$.

Submit: `RecursiveSum.asm` and the translator `VMTranslator5.py`.

Hack Assembly-Language Reference

Memory and predefined symbols

The Hack CPU has registers `A` and `D`. The symbol `M` means `RAM[A]`. Thus, for example, `D=M` copies `RAM[A]` into `D`. Predefined symbols include `R0`–`R15`, with `SP=R0`, `LCL=R1`, `ARG=R2`, `THIS=R3`, and `THAT=R4`.

A-instruction

Form	Meaning
`@value`	Set `A` to a nonnegative decimal value.
`@symbol`	Set `A` to the address represented by a predefined symbol, label, or variable.

Examples: `@256`, `@SP`, `@R10`, and `@LOOP`.

C-instruction

The general form is `dest=comp;jump`. Both `dest=` and `;jump` are optional. The computation is performed first; its value is stored in every named destination, and the jump is decided using that same value.

Field	Allowed forms	Meaning
`dest`	null, `M`, `D`, `MD`, `A`, `AM`, `AD`, `AMD`	store the computed value
`jump`	null, `JGT`, `JEQ`, `JGE`, `JLT`, `JNE`, `JLE`, `JMP`	jump to ROM[`A`]

All allowed `comp` forms					
`0`	`1`	`-1`	`D`	`A`	`M`
`!D`	`!A`	`!M`	`-D`	`-A`	`-M`
`D+1`	`A+1`	`M+1`	`D-1`	`A-1`	`M-1`
`D+A`	`D+M`	`D-A`	`D-M`	`A-D`	`M-D`
`D&A`	`D&M`	`D|A`	`D|M`		

Jump conditions test the computed value: `JGT` > 0, `JEQ` $= 0$, `JGE` ≥ 0, `JLT` < 0, `JNE` $\neq 0$, `JLE` ≤ 0, and `JMP` always jumps.

Useful examples

Assembly	Effect
`@256 / D=A / @SP / M=D`	Set `SP` to 256.
`@SP / A=M / M=D`	Store `D` at the next free stack location.
`@SP / M=M+1`	Increment `SP`.
`@SP / AM=M-1 / D=M`	Decrement `SP`, address the popped value, and copy it to `D`.
`@LOOP / 0;JMP`	Jump unconditionally to `(LOOP)`.
`@TARGET / D;JNE`	Jump to `(TARGET)` if `D` is nonzero.

Labels are declared as `(LABEL)` and do not occupy a ROM instruction address. Comments begin with `//`.

Python Quick Reference

This reference contains the basic Python operations needed to read a VM file and write the corresponding assembly file.

Command-line argument and output filename

If the program is run as `python VMTranslator1.py Program.vm`, then `sys.argv[1]` is the string `"Program.vm"`.

```
import sys

input_name = sys.argv[1]
output_name = input_name.rsplit(".", 1)[0] + ".asm"
```

Opening and reading a text file

Using `with` closes the file automatically when the block finishes.

```
with open(input_name, "r") as source:
    lines = source.readlines()       # list containing all input lines

with open(input_name, "r") as source:
    for line in source:              # process one line at a time
        print(line)
```

Removing comments and splitting a VM command

```
code = line.split("//", 1)[0].strip()
if code == "":
    continue                         # ignore blank/comment-only lines

parts = code.split()                 # e.g. ["push", "constant", "17"]
command = parts[0]
segment = parts[1]
value = int(parts[2])                # convert the string "17" to 17
```

Building and writing the output

```
assembly = []
assembly.append("@256")
assembly.append("D=A")
assembly.extend(["@SP", "M=D"])

with open(output_name, "w") as output:
    for instruction in assembly:
        output.write(instruction + "\n")

# Alternatively:
with open(output_name, "w") as output:
    output.write("\n".join(assembly) + "\n")
```

Useful Python statements

```
if command == "add":
    # translate add
    pass
elif command == "push":
    # translate push
    pass
else:
    print("Unknown command:", command)

for item in parts:
    print(item)
```

```
def make_label(prefix, number):
    return prefix + str(number)

counter = 0
label = make_label("TRUE", counter)
counter = counter + 1

address = 5 + value
text = f"@{address}"   # formatted string
```

Useful list and string operations include:

Python	Meaning
`len(items)`	number of items in a list or characters in a string
`items[i]`	item at position `i`; indexing starts at 0
`items.append(x)`	add one item to the end of a list
`items.extend(xs)`	add all items from another list
`text.strip()`	remove whitespace from both ends of a string
`text.split()`	split a string at whitespace
`str(number)`	convert a number to a string
`int(text)`	convert a decimal string to an integer

End of question paper